Ain Shams Engineering Journal 15 (2024) 102478

Contents lists available at ScienceDirect

Ain Shams Engineering Journal

journal homepage: www.sciencedirect.com

A novel algorithmic Data-Collision SDRAM-based TCAM architecture on FPGA

Nguyen Trinh [a,b], Minh Bui [a,b], Binh Dang [a,b], Linh Tran [a,b,*]

[a] Department of Electronics Engineering, Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City, Viet Nam
[b] Vietnam National University, Ho Chi Minh City, Viet Nam

ARTICLE INFO

Keywords:
Ternary Content-Addressable Memory
Data-Collision TCAM
FPGA
SDRAM-based TCAM

ABSTRACT

Ternary Content-Addressable Memory (TCAM) is one of the most effective methods for high-speed data searching in networking infrastructure. Despite the excellent performance and large look-up table, application-specific integrated- circuit-based TCAMs cost massive power supply and are challenging to configure. Field-Programmable Gate Arrays (FPGAs) have become the alternative solution for integrating TCAM due to their minimal power consumption and reconfigurability. Methods using on-chip registers, BRAMs, or LUTRAMs to implement large-size TCAM suffer excessive resource consumption. Researchers introduced DDR-SDRAM-based CAM to provide high-performance and economic packet inspections to overcome such issues; however, they only support binary look-up. This paper presents a novel algorithmic Data-Collision TCAM architecture on FPGA for SDRAM compatibility. The proposed SDRAM- based TCAM supports ternary value look-up function for practical Ethernet packet processing. The architecture outperforms conventional TCAMs regarding BRAMs, logics consumption, and look-up table sizes. The proposed architecture is the first SDRAM-based TCAM on FPGA with a 128Mbyte look-up table.

1. Introduction

Content-Addressable memory (CAM) is a special memory that searches the input data with the entire preloaded database and generates the corresponding address information [1,2] benefiting routing Internet packages or controlling a massive database. There are two categories of CAMs: binary CAM (BCAM), supporting only two states of value (‘0′ and ‘1′), and ternary CAM (TCAM), supporting an additional wildcard state (‘0′, ‘1′, and ‘X’) with ‘X’ value as don’t care bit or ternary values. BCAM can only search for the exact data [3], but TCAM can look up ternary values, making it a vital point for improvement in the digital industries [4]. An exemplary of TCAM implementation as lookup table shows a database locates inside TCAM architecture which stores the values and the rules to which the data belongs. TCAM compares the incoming data (“1 1 1 0”) with possible matching values and chooses the appropriate rule (Rule C), as illustrated in Fig. 1. However, the limitation is that to perform this ability, TCAM hardware has to load all storage cells whose width and depth are fixed during searching operation and consumes a considerable amount of power [5,6].

Field Programmable Gate Arrays (FPGAs) are famous for their reconfig- urability and low power consumption, making them highly favorable for the development of emerging memory-based computing [7,8,9]. Modern FPGAs data acquisition boards (DAQ) have even integrated off-chip memory, such as SDRAM, to support FPGA-purposed applications [10,11]; however, they still lack an efficient TCAM core because of its restriction to specific applications that would not be widely used. This encouraged many researchers to integrate their TCAM architecture into FPGAs [12,13,14,15,16,17,18] over the years. The development of FPGA-based TCAMs in recent years has been slowed down and stuck with using registers, block RAMs, or distributed-RAMs al- gorithms [2] rather than using different types of memory implementation to emulate TCAMs and to improve performance. These implementations signifi- cantly consume FPGA’s resources to achieve TCAM functionality, which barely left users any resources for further upgrades. Many researchers have proposed alternatives TCAM architecture to utilize hardware resources of FPGA when integrating a TCAM [7,12]; but memory usage efficiency on FPGAs is still a problem.

This paper proposed an SDRAM-based TCAM implementation on FPGA that uses the Data Collision algorithm. We exploit the advantages of the exter- nal SDRAM that help FPGA-based TCAM achieve better

* Corresponding author at: Department of Electronics Engineering, Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City, Viet Nam.
E-mail addresses: nguyentvd@hcmut.edu.vn (N. Trinh), minh.buiminh5052vn@hcmut.edu.vn (M. Bui), binh.dang.eboy@hcmut.edu.vn (B. Dang), linhtran@hcmut.edu.vn (L. Tran).

https://doi.org/10.1016/j.asej.2023.102478
Received 28 March 2023; Received in revised form 2 July 2023; Accepted 24 August 2023
Available online 14 September 2023

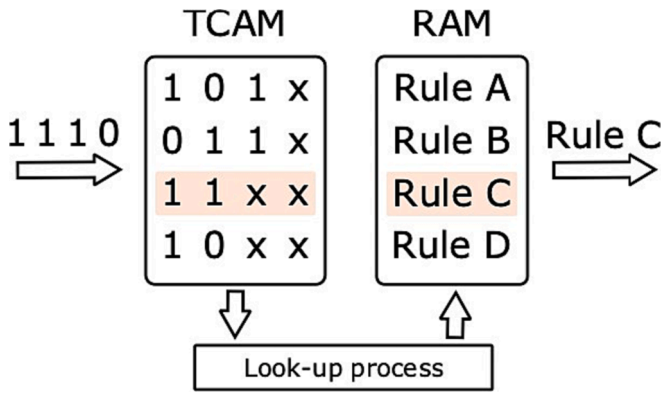

Fig. 1. RAM-based TCAM implementation of a lookup table.

efficiency in utilizing hardware resources inside FPGAs with large-size reconfigurable table lookups while maintaining high-speed searching time. The proposed design allows de- velopers to easily integrate TCAM onto small FPGAs. The advancements are the capacity of a 128Mbyte lookup table while sustaining high-speed searching time compared to the state-up-the-art ASIC-based and FPGA-based TCAMs [7,19,20,21]. The architecture in the paper is an improvement [20] in terms of data storage and capacity.

The rest of the paper is structured as follows: Section 2 discusses related works and provides the proposed design's motivations and key contributions. Section 3 describes the proposed TCAM methodology. Section 4 demonstrates the proposed hardware implementation. Section 5 demonstrates the feasibility of the proposed design and comparisons with other TCAMs architecture. Section 6 draws conclusions of this work and discusses future developments.

2. Motivations and contributions

2.1. Preliminaries

Over the years, emulated TCAMs on FPGA has become a mature research area and consists of numerous of different architectures collectively known as FPGA-based TCAMs. According to [2], FPGA-based TCAMs are classified into three categories: registered-based TCAMs, BRAM-based TCAMs, and dis- tributed RAM-based TCAMs.

Roth et al. [19] introduced an advanced application-specific integrated- circuits TCAMs (ASIC-based TCAMs) that had the capacity up to 9Mbits, which can store up to 128 K 72-bit word, pushing the limit of conventional TCAMs at that time. But because of the low reconfigurability and challenging integration with FPGA-based systems [22,23], conventional TCAMs are out of favor, which paves ways for emulated TCAMs on FPGA to be a better option to integrate searching applications.

One of the first approaches of emulated TCAM was using SRAMs which combines additional logic to replicate the TCAM's functionality. A candidate called E-TCAM [15] is a model which reduces the number of block-RAMs, slice registers, and LUTs and improves searching speed compared with the best avail- able SRAM-based TCAM designs. Despite its advantages, the E-TCAM imple- mentation consumes high FPGA on-chip resources with low reconfigurability, making integrating them into large-scale designs or systems hard. Another candidate is Pseudo-TCAM [13] with low area cost and high throughput yet requiring unique mapping and relocation for updates. Multiple Hash Matching Units use embedded block RAMs to create efficient resource usage and scalable TCAM but struggle with hashing collision [22].

Apart from the abovementioned categories, there are emerging trends in implementing TCAMs using external memories. One is SDRAM-based TCAMs using SDRAM to store TCAM rules. The SDRAM-based TCAMs increase table size significantly due to their high-density feature but negatively affect performance in terms of clock cycles because of addressing operation in SDRAM.

A candidate called SDRAM-based Hash-CAM [23] proved the concept of using DDR-SDRAM to emulate CAM, which shows great advancement in table size. However, it does not support the ternary value 'X' and still requires a certain number of block RAMs to store collided data for full functionality; thus, it is restricted to 64 K 32-bit word storage capacity.

Our approach is based on the fourth category: SDRAM-based CAM/TCAM. All types of FPGA-based TCAM are part of our comparisons in Section 5.

2.2. Motivations

TCAM is one of the critical new components in modern networking systems, operating as a high-speed routing table for packet classifying and forwarding, but the conventional CAM/TCAM requires significant power dissipation with high area cost and low scalability. Therefore, FPGA-based CAM/TCAM has become favorite due to its easy integration and wide application.

Irfan et al. [2] asserted that three majors types of FPGA-based CAM/TCAMs have their trade-offs. SRAM-based TCAMs were not implementable on FPGA resources. Current data partitioning and mapping algorithms make it possible to be implemented on FPGA. However, it still struggles with increasing lookup table size because of the capacity limitations of on-chip resources and collision probability. Therefore, the current FPGA-based CAM/TCAMs do not beat the conventional CAM/TCAMs but rather offer reconfigurability feature. Thus, the future developments of CAM should focus on efficiency in terms of hardware resources, power consumption, and throughput.

An alternative is to use the high-capacity SDRAM to overcome the above- mentioned challenges. In this work, we improve the algorithm [20] in terms of SDRAM compatibility functions and pave the way for future developments of large-scale FPGA-based TCAM without the restriction of RAMs or registers. We propose a novel TCAM hardware architecture that uses external SDRAM as data storage without the restriction of RAMs. The architecture effectively utilizes hardware resources on FPGA and shows significant advancement in lookup-entrant capacity thanks to the exploitation of the high-density feature of SDRAM.

2.3. Key contributions

Key contributions of the proposed work are as follows:

- A novel SDRAM-based TCAM architecture outperforms ASIC- and FPGA- based TCAMs regarding table size. It is state-up-the-art for 256 K lookup entrants for 72-bit data, doubled in size compared to advanced ASIC- based 9Mbit TCAM [19]. In addition, the new implementation shows improvements for 104-bit data from 256 to 8-Thousand-Entrants lookup when using SDRAM instead of block RAMs [20].
- The SDRAM-based Data-Collision TCAM shows significant advancement in cutting off on-chip block RAM usage by 100% while sustaining less than 5% LUTs and registers consumption due to the data migration to SDRAM. With this resource consumption, it is now possible for small FPGA devices that support DDR-SDRAM to fit a large-size TCAM.

3. Proposed methodology

3.1. SDRAM as data storage

RAM-based TCAM s' table size is modest due to the limited address and data width of RAMs; thus, current FPGA-based TCAM architectures have difficulty storing large databases.

Modern SoC FPGA devices powered by Altera contain an SDRAM chip with a capacity of 4 GB [24], comprised of two x16 DDR3 SDRAM chips for the storage module of a high-speed data acquisition board [11]. The hard processor system (HPS) SDRAM controller subsystem provides efficient access to external SDRAM for the MPU subsystem, the L3 interconnect, and the FPGA fabric [24], as in Fig. 2. It offers port and bus configurability, error correction, and power management for external memory.

Fig. 3 shows the Cyclone V SoC Development Kit HPS Physical Memory Map [24] with un-decoded regions. Since L3, MPU, and FPGA fabric share the same 4 GB SDRAM region, careless access to FPGA address space can fatally affect the L3 or MPU systems while functioning. Thus, it is better to allocate a secure access region for FPGA fabric without affecting the functionality of the L3 and MPU on HPS. Therefore, we configure the boot region, usually used for the Linux operating system, to use the bottom 512 MB memory in this implementation. The remaining upper 512 MB is used by FPGA fabric.

3.2. Data arrangement methodology

We add extra bits into the data structure to present the ternary values, known as mask vectors. Each rule has its own mask vector that marks the corresponding ternary values in the key string; each bit in the vector indicates a particular fragment containing ternary values 'X.'

In the proposed method [20], the system cuts and stores rule id in one storage location while the corresponding mask of the key data is stored at another location, as illustrated in Fig. 4 due to the limited storage size of the SRAM. With the massive storage size of SDRAM, however, instead of just writing rules into memory, the mask vector information is included along with the corresponding rule in the data structure to search for match vectors later. With each key input for setting, the system cuts, concatenate it with a corresponding mask, and stores it at the same location. Thanks to this data structure, we significantly reduce the number of resources for containing masks and enhance the data processing speed compared to the older version.

Fig. 5 shows the arrangement of data in the database using the conventional RAM-based technique. We take an example of a 10-bit key string divided into 5 2-bit pieces, so-called fragments. RAMs are connected in parallel for data accommodation; each RAM indicates a segment mat. Each key fragment ad- dresses a particular segment mat to store the rule id, while the rule id addresses the confirmation database RAM to store the key string for later search.

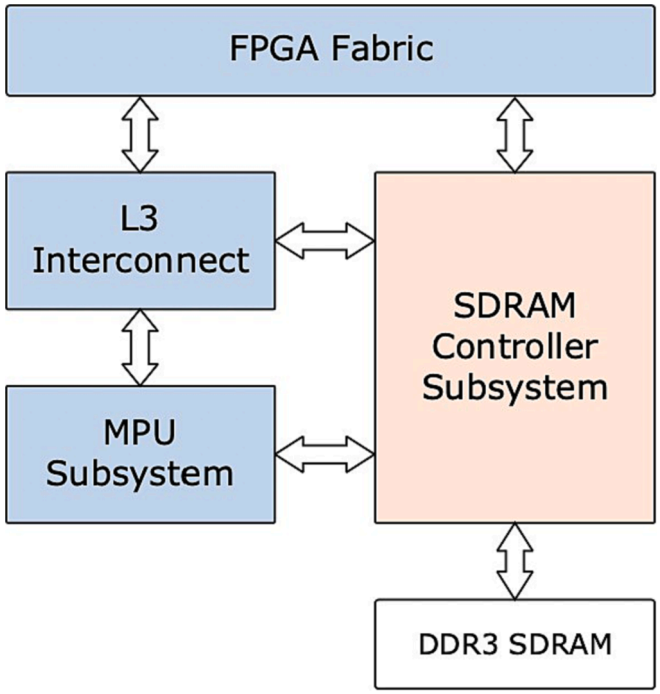

Fig. 2. The schematic diagram of the HPS system integration.

Since SDRAM is a unified memory for the whole system, it is impossible to accommodate data in such a conventional way [20]. The linear chaining technique is adapted to store data into SDRAM effectively. This technique helps the system perceive the unified memory as a group of segment mats. Segment mats are chained to one another instead of connecting in parallel. After allocating all segment mats, the remaining memory space is for the confirmation database to accommodate. Fig. 6 shows how the new rule id is distributed into the SDRAM. The exact 10-bit key string is used to illustrate the process. Each color represents a segment mat. Each fragment is assigned to a particular segment mat with the corresponding color; hence, the rule id accommodates within that region. For example, with 2-bit fragments, each mat has four locations; each location is corresponded to 00, 01, 10, and 11, respectively, starting from the first location within each mat. The white region is the remaining address space for the confirmation database. The rule id addresses this region to store the key string for a later search stage.

Fig. 7a shows how the system partitions 512 MB SDRAM into chained regions to fit in the TCAM database. Segment mats are stacked, starting from the bottom at 0x0 address. The database for confirmation locates at the top of the memory window. Therefore, it is necessary to consider the size of the confirmation database to decide the proper size of each segment mat.

Take a 10-bit key string with five 2-bit fragments as an example. The base addresses (in hex) for each memory region inside the SDRAM are shown in Fig. 7b.

3.3. Searching for address

In this implementation, the Avalon-MM interface is used to dispatch setting and searching operations. It allows single read and write from and to SDRAM only; simultaneous access is not supported. Thus, a new searching algorithm is necessary to reduce the search time of the system.

Fig. 8 illustrates the process of looking for rules from memory. First, a rule vector can already be found by extracting data corresponding to each fragment, however, different processes are needed to produce accurate rules from key strings. As mentioned, mask information for each rule included along with the rule in the data structure, is used to apply to the particular fragment of the key string. This masking process only involves written rules, meaning the extracted segment is neither empty nor in a collision state. Next, the masked key input string is compared with the original one. Masked fragments can be bypassed, and non-masked ones can be compared with the rule's original key string. All confirmation results are prioritized in the final stage to generate the final address output.

4. Proposed architecture

4.1. Hardware system architecture

Fig. 9 shows the general hardware architecture diagram of the design. Data input goes through a straightforward process to produce the corresponding address at the output. The architecture supports two modes: setting and searching.

In setting mode, the segmentation module starts cutting the key string into fragments and uses these fragments to address SDRAM to distribute the necessary data into the database iteratively.

The segmentation module repeats the partitioning process in searching mode to extract rule id from SDRAM. The extracted rules then go through a masking process. Based on the attached mask information, the process eliminates collided and empty rules; only written ones can go on to the next step. Finally, confirmation and address selection

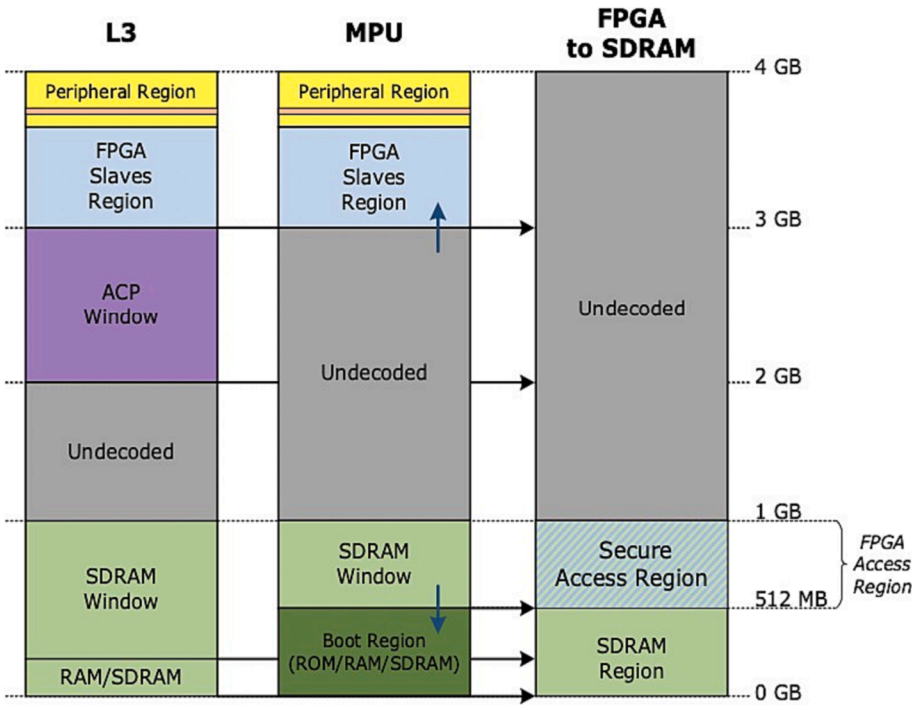

Fig. 3. FPGA-to-SDRAM secure access region.

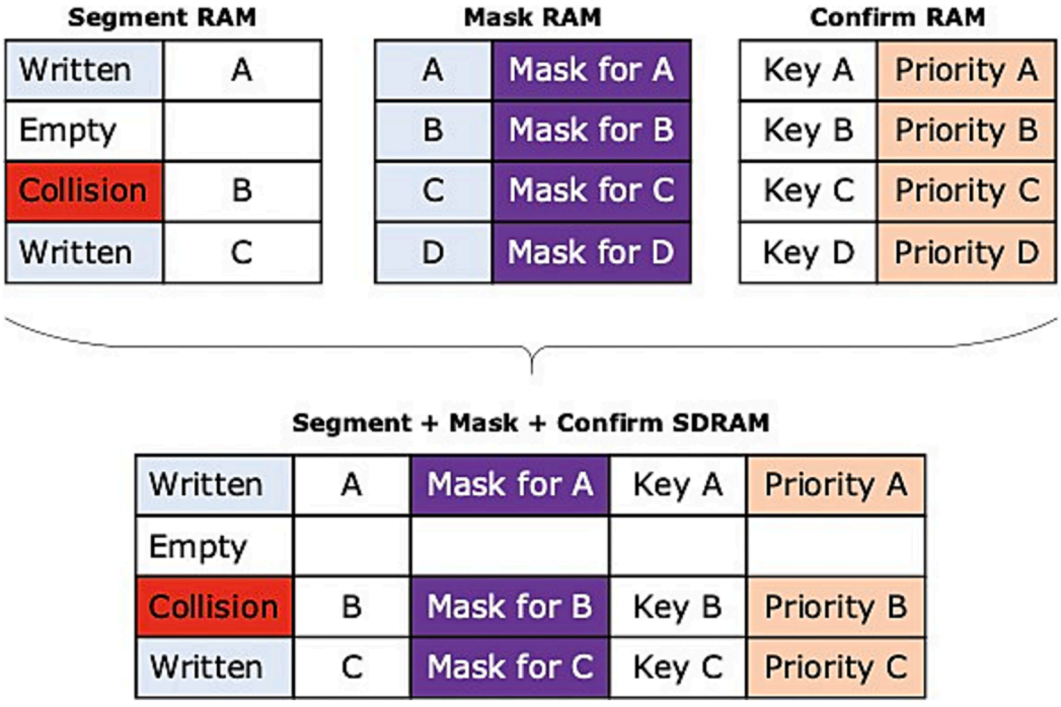

Fig. 4. All information now included in the database.

compare the masked key input and the corresponding rule's original key to give accurate results.

4.2. Hardware design on FPGA

4.2.1. Segmentation module

The key strings are first sent to the segmentation module to make cut-points and partition SDRAM for setting and searching operations. A controller then enters setting or searching mode as soon as it receives a set or search flag. The hardware design of the segmentation module is shown in Fig. 10.

In setting mode, the Segmentation module ensures that it partitions the SDRAM correctly into the aforementioned chained order and that segment mats do not conflict with their address spaces. The module consists of a defined- pattern counter which continuously produces address prefix values. This value is appended to each fragment to form a proper TCAM address. The module applies algorithm 1 to populate the TCAM database into SDRAM. The algorithm's variable' prefix'

represents the prefix that allocates the SDRAM into different address spaces. Variable 'i' represents the corresponding key fragments, together with 'addr prefix,' both form a'tcam address'. After forming an accurate address, i.e.,'ruleid' for the memory, the module extracts data at that particular location, based on which the new data is modified before being written into SDRAM. The process repeats till it runs out of fragments. Then, it forms the address for the confirm-database region and stores the key string.

The searching operation iterates the fragmentation process producing proper'tcam address' for data extraction. According to the address information, the corresponding segments are extracted from the SDRAM for further processing. Algorithm 2 shows how the Segmentation module iterates through the memory to get segment outputs.

Since the Segment Engine iterates the reading and writing process of SDRAM based on the number of fragments, the complexity of the algorithm for both set- ting and searching is O(n) where n notates the number of fragments.

Algorithm 1: Setting Rule ID

```
Input: Key information
Output: SDRAM-based TCAM Table
/* Function declaration */
Function Fragmentation(prefix, width, key):
    address = (key ≪ width);                    // cut key by shifting out
    return tcam_address = {prefix, address};    // concatenation
Function Modify(set data, cur segment):
    {set_ruleid, set_mask, set_key, set_priority} = set_data;
    {cur_status, cur_ruleid, cur_mask, cur_key, cur_prio} = cur_segment;
    if (cur_status indicates EMPTY) then
        mod_status = WRITTEN;
        mod_ruleid = set_ruleid;
        mod_mask = set_mask;
        mod_key = set_key;
        mod_priority = set_priority;
    else if (cur_status indicates WRITTEN or COLLIDED) then
        mod_status = COLLIDED;
        mod_ruleid = cur_ruleid;
        mod_mask = cur_mask;
        mod_key = cur_key;
        mod_priority = cur_priority;
    return mod_segment = {mod_status, mod_ruleid, mod_mask,
                          mod_key, mod_priority};
/* Main: TCAM Setting */
Function Main:
    W_fragments = W_key / n_fragments;
    addr_prefix = 0;
    for i ← 0 to i ← n_fragments do
        // generate TCAM address and lookup the table
        tcam_address = Fragmentation(addr prefix, W_fragments, key);
        cur_segment = SDRAM[tcam_address];      // lookup TCAM table
        // modify segment for appropriate setting
        mod_segment = Modify(set data, cur_segment);
        // store modified segment into the database
        SDRAM[tcam_address] = mod_segment;
        // increment the address prefix
        addr_prefix = addr_prefix+1;
    return setting_done;
```

Algorithm 2: Searching for Rule ID

```
Input: Searching Key
Database: SDRAM-based TCAM Table
Output: Segments (Status & Rule ID)
/* Function declaration */
Function Fragmentation(prefix, width, key):
    address = (key ≪ width);                    // cut key by shifting out
    return tcam_address = {prefix, address};    // concatenation
/* Main: TCAM Searching */
Function Main:
    W_fragments = W_key / n_fragments;
    addr_prefix = 0;
    for i ← 0 to i ← n_fragments do
        // generate TCAM address and lookup the table
        tcam_address = Fragmentation(addr prefix, W_fragments, key);
        segment_out = SDRAM[tcam_address];      // lookup TCAM Table
        // increment the address prefix
        addr_prefix = addr_prefix+1;
    return searching_done;
```

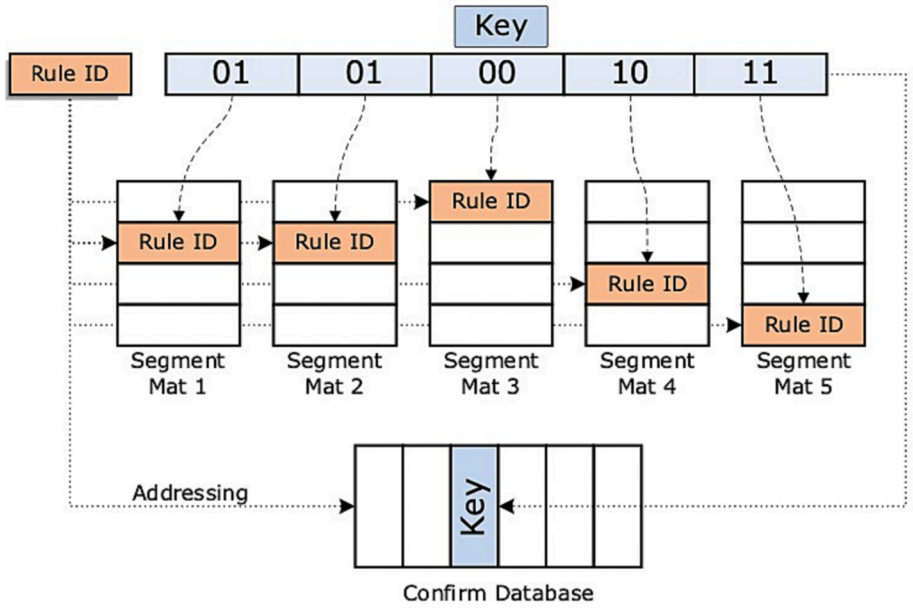

Fig. 5. Conventional data arrangement.

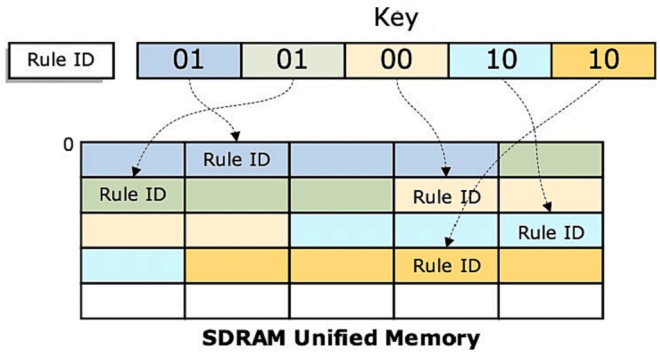

Fig. 6. Segment mats and confirm database are chained to each other.

4.2.2. *Masking module*

As the Segmentation module extracts segments from SDRAM sequentially, there needs to be a way to handle the data to save search time effectively. The mask module uses the data queueing processing technique in its design. It consists of multiple pipelines to receive and process every data from the SDRAM controller sequentially. As a result, the module always produces results within a certain number of cycles. Fig. 11 illustrates how the masking module takes and produces data. The module is designed as a pipelined queue that masks the key string based on the corresponding rule's mask information.

Fig. 12 illustrates the hardware design of the mask module. Before the masking process, the module detects and eliminates all the segments in collided or empty states; only valid ones can be bypassed. Also, repeating rules are ignored to optimize search time and reduce switching activities.

4.2.3. *Confirmation module*

The confirmation process ensures that the input key string matches the rule database. The module also implements the data queueing technique to optimize the search time.

The confirmation module receives the rule id with its corresponding masked key, and the appended key input for comparison. The masked key then compares with the appended input key to give the final result. If two key strings match, it appends the rule ID with a priority number for later address selection. Fig. 13 describes the hardware architecture of the confirmation module. If two key strings do not match, the module eliminates the current rule and waits for the next one; thus, the confirmation module always produces matched rule ids with their priority numbers.

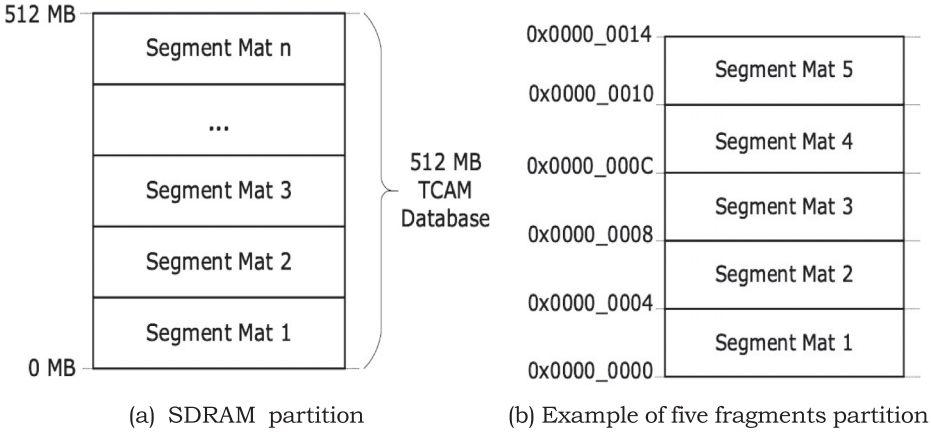

Fig. 7. The architecture partitions SDRAM into segment mats a. SDRAM partition scheme, b. Example of five segment mats partition.

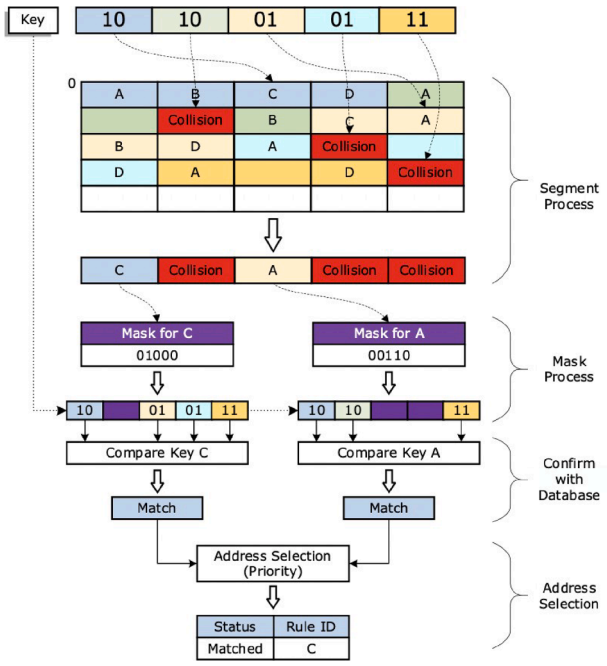

Fig. 8. Searching procedure.

4.2.4. *Priority module*

Address or priority selection is the final stage of searching for rule id. It prioritizes rules to be presented at the output. The priority module compares the values attached to each matched rule id and gets the highest-priority match. The result is the rule id that matches the key. If no rule matches the key string, the module informs that the key does not match.

The hardware design of the priority module is illustrated in Fig. 14. The module uses a feedback mechanism when comparing priority numbers as it receives rule ID inputs sequentially.

4.2.5. *SDRAM controller interface module*

Like the segmentation module, the SDRAM controller interface is vital in implementing an SDRAM-based TCAM. The module forms a bridge between the whole architecture and SDRAM. The module supports the initialization function besides read and write operations. The module contains a state machine that is responsible for any communications between the FPGA hardware and SDRAM. The state machine for reading, writing, and initializing the SDRAM is shown in Fig. 15.

The FPGA-to-HPS SDRAM interface exposes the entire 4 GB address space to the FPGA fabric. As mentioned in Section 3, we want our TCAM to access the upper 512 MB secure region without failing other parts of the chip. But, it is hard for the system to effectively control the lower and upper bounds of the region effectively.

The Cyclone V FPGA devices provide an Address Span Extender component that offers a common memory management technique called paging [26]. Fig. 16 shows how we implement the IP into our system. The IP component provides a memory-mapped window into the upper 512 MB that our TCAM masters. Using the address span extender, the system always safely accesses the provided address space without affecting other regions.

5. Implementation and evaluation

The hardware design of SDRAM-based TCAM is built on the FPGA Cy- clone V DE-10 Standard device (5CSXFC6D6F31C6) with Hard Processor System's ISSI SDRAM memory models for functional verification. The design is intended to be built on a small FPGA development kit such as the Cyclone V SX (5CSXFC6D6F31C6) device to prove that our architecture successfully utilizes the high-density feature of SDRAM

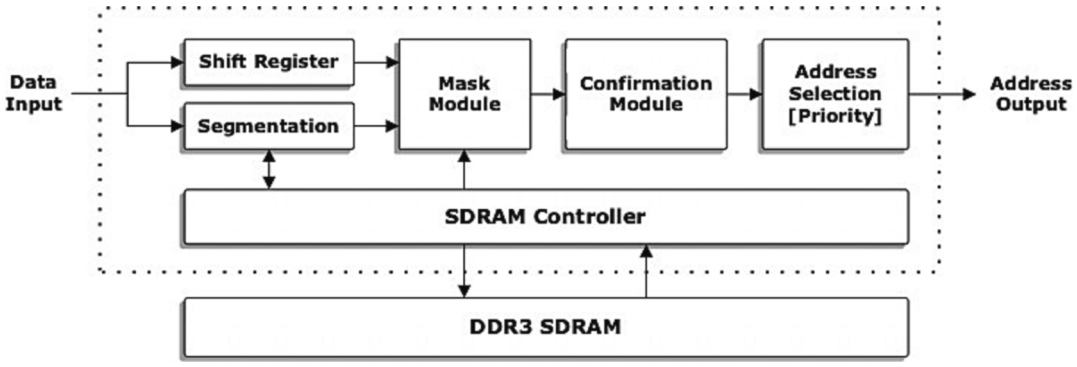

Fig. 9. Overall system architecture.

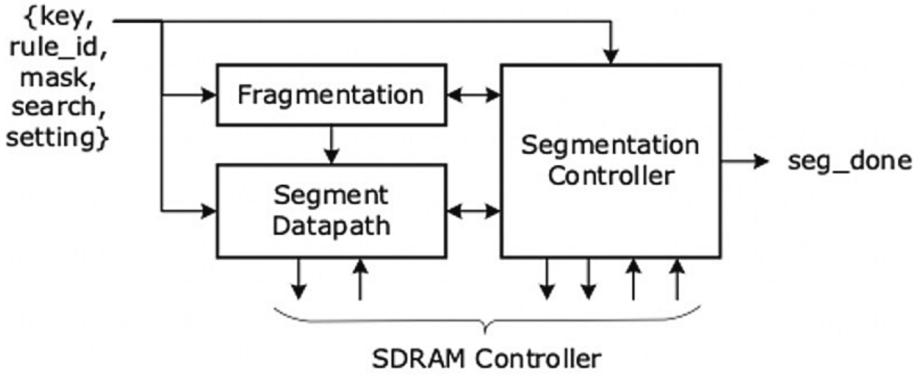

Fig. 10. Hardware design of the segmentation module.

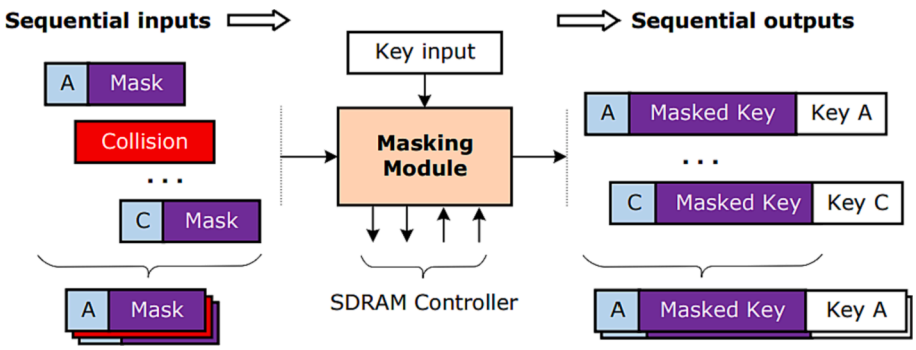

Fig. 11. Mask module takes and produces data sequentially.

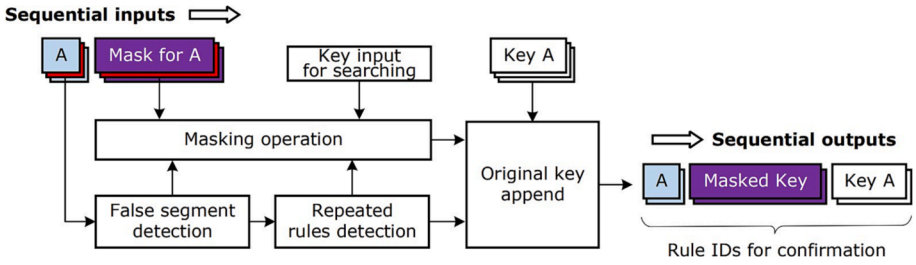

Fig. 12. Hardware design of the mask module.

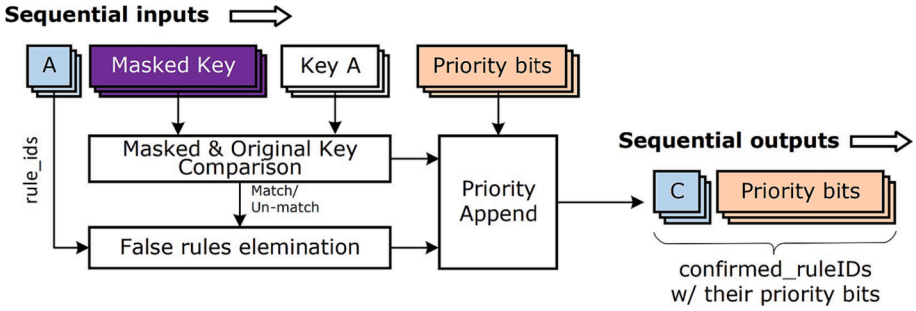

Fig. 13. Hardware design of the confirmation module.

without consuming too much FPGA on-chip's resources. We discuss and evaluate the proposed design in this section.

5.1. Data allocation

The SDRAM usage can be calculated using equations (1), (2) and (3).

$$U_{SDRAM} = 2^{AW_{DDR\_SDRAM}} * DW_{DDR\_SDRAM} \quad (1)$$

$$AW_{DDR\_SDRAM} = \left[\log_2\left(n^{frag+1}\right)\right] + \left(\left[\frac{W_{data}}{n_{frag}}\right]\right) \quad (2)$$

$$DW_{DDR\_SDRAM} = 2 + W_{rule\_ID} + W_{confirm\_key} + W_{mask\ ID} + W_{prio} \quad (3)$$

where U_{SDRAM} denotes the *SDRAM* usage in bits; $AW_{DDR\ SDRAM}$ and $DW_{DDRSDRAM}$ are the address and data widths of *SDRAM*. W_{data} is the width of key data; n_{frag} is the number of fragments defined by the users; $W_{rule\ ID}$, $W_{segment}$, and $W_{confirm\ key}$ represent the width of rule IDs, segments, and confirmation data, respectively.

The Address Span Extender component only supports a specific range of SDRAM's depths and widths. Tables 1 lists the relationship between SDRAM's address and data width configurations. Equations (1), (2), and (3) must satisfy the below constraints to calculate SDRAM size correctly.

The maximum of fragments' bits determines the depth of TCAM to keep the collision ratio under-entry [25]. Tables 2 lists the memory utilization regarding the number of key entries and data widths configurations.

The implementation remarkably proves its superiority in increasing table size by using SDRAM as data storage. It can store up to 8 K lookup entrants of 104-bit data compared to the moderate number of 256 entrants in the pro- posed RAM-based methodology [20]. Moreover, this TCAM can look up 256- Thousand entrants for 72-bit data configuration, considerably more extensive than the cutting-edge FPGA-based 2.4Mbits TCAM [26]. The implementation doubles in size compared to ASIC-based 9Mbits [19], which has 128 K table entries at most.

The Data-Collision algorithm is efficient in resource saving. A 256 K lookup circuit with 144-bit requires 135 MB, only 25% of the total memory space. The algorithm also shows great scalability since the number of key entries is nearly independent of memory space allocation as the fragment's bits decide the depth of TCAM. The fragment's bits must be maximum to fit as many entrants as possible. For instance, the 104-bit data supports a maximum of 13-bit per fragment; while for the 128-bit key string, the length is 16-bit per fragment; and the 144-bit key supports 18-bit long.

5.2. Implementation evaluation

This work selects a 64Kx128-bit TCAM table a 24-bit address with 256-bit data SDRAM controller. The Tables 3 presents the synthesis

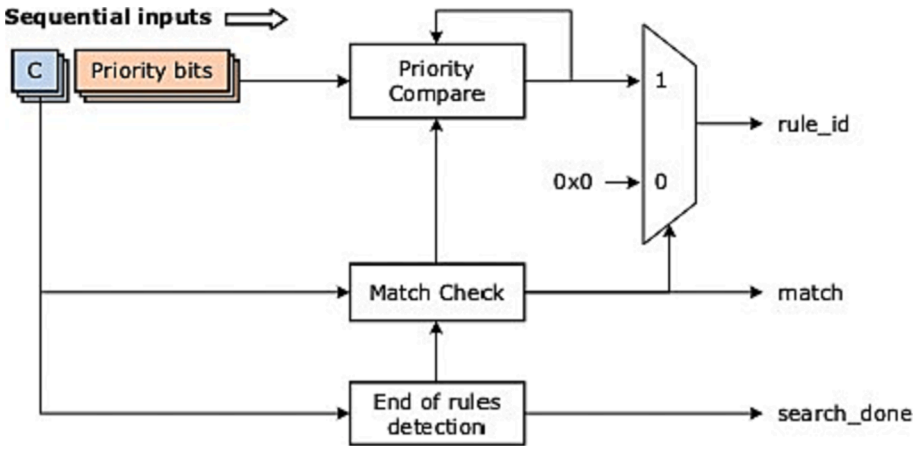

Fig. 14. Hardware design of the priority module.

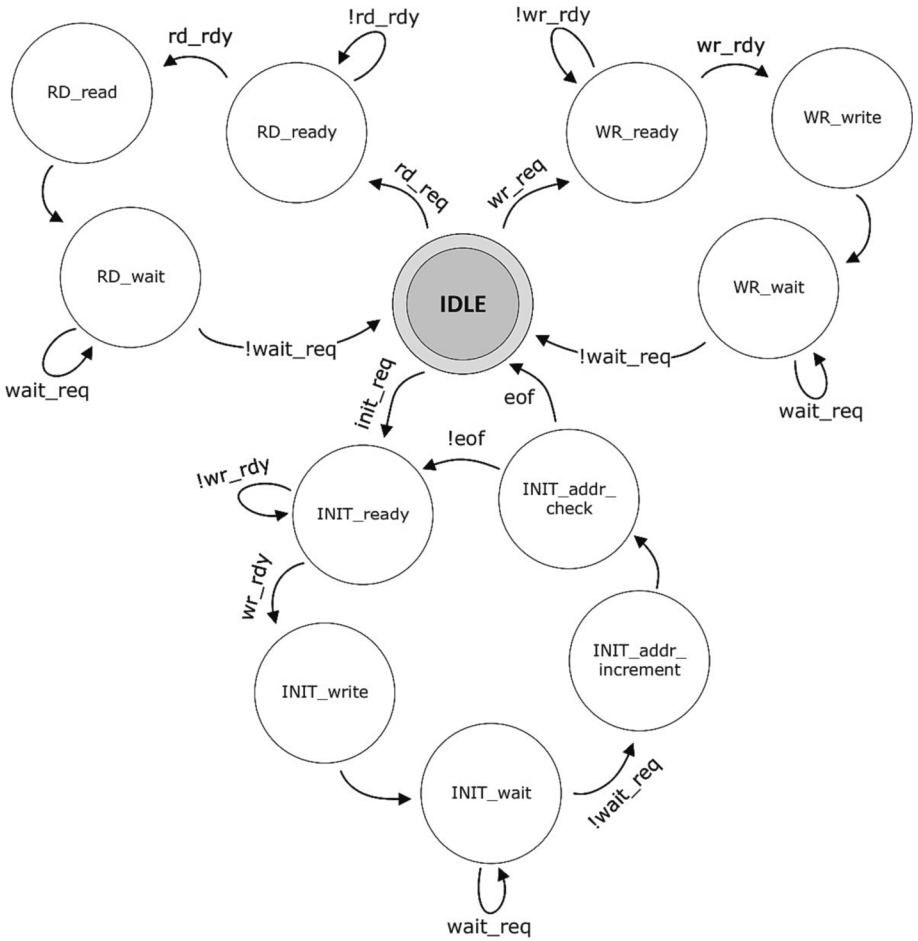

Fig. 15. SDRAM operations state machine.

results of the whole system. The architecture shows excellent efficiency in utilizing on-chip logic. The entire system has the capability of 64-Thousand IPv6 packet lookups while consuming less than 5% of logic in the selected devices.

Tables 4 compares multiple cutting-edge FPGA-based TCAMs. The result shows that our novel SDRAM-based TCAM architecture remarkably saves on- chip memory consumption due to the data emigration to SDRAM while sustaining equivalent or even lower registers usage. On the other hand, the related work [23] still requires a significant number of memory bits. However, the average search time in nanosecond of the architecture is the lowest due to the iterative communication between the Segment Engine and the SDRAM Controller Interface.

Due to the SDRAM-iteration process in the segmentation stage, the through- put of the architecture is inversely proportional to the number of fragments. In this implementation, the time for a read operation of SDRAM is simulated as 3 cycles per request. For a packet size of 84 bytes, in the 64 K $\times$ 36-bit configuration, the system's throughput is moderate at 278.3 Kpps, which is suit- able for under 200Mbps Ethernet forwarding interfaces. At the same time, the 64 K $\times$ 128-bit can produce 378.6 Kpps with the most optimal configuration.

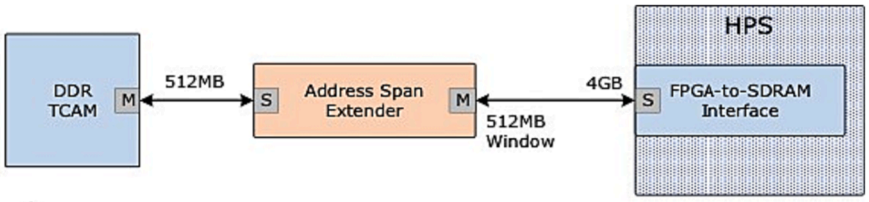

Fig. 16. The address span extender component.

Table 1
SDRAM's depth and width configurations.

AWDDR SDRAM	28	27	26	25	24	23	22
DWDDR SDRAM	16	32	64	128	256	512	1024

Table 2
Estimated memory allocation percentage of different configurations.

Key Entries	Data widths				
	36-bit	72-bit	104-bit	128-bit	144-bit
8 K	1.56%	3.13%	0.39%	3.13%	12.5%
16 K	1.56%	3.13%	–	3.13%	12.5%
32 K	1.56%	3.13%	–	3.13%	12.5%
64 K	1.56%	3.13%	–	3.13%	12.5%
128 K	1.56%	3.13%	–	–	12.5%
256 K	1.56%	3.13%	–	–	12.5%

Table 3
Synthesis of SDRAM-based Data-Collision TCAM on Cyclone V FPGA (5CSXFC6D6F31C6).

	ALUTs	Registers	Memory	PLLs	DLLs
Number	1,385	3,705	0	1	1
Percent (%)	3%	5%	0%	7%	25%

6. Conclusion and future work

Implementing TCAMs using state-up-the-art FPGAs is never out of interest for TCAM's importance in networking systems. However, most of the existing FPGA-based TCAM designs have difficulty scaling up to support large-size table lookups, mainly due to the constraints in algorithms and on-chip resources.

This paper successfully developed and implemented a novel algorithmic Data- Collision SDRAM-based TCAM hardware architecture. The design can be integrated into any Intel FPGA devices that support SDRAM. We constructed several formulas and analyzed resource utilization and performance thoroughly. We comprehensively evaluate our design to understand various performance and resource utilization trade-offs. The FPGA implementation results show that our design can support up to 128Mbyte TCAM, a very high-density TCAM while sustaining low resource consumption and great configurability. Our first SRAM-free FPGA-based TCAM paves the way for an era of integrated low- cost reconfigurable, and high-density TCAM that can genuinely compete and gradually replace the hardware version in multiple systems and research onward. However, the architecture can only support up to 200Mbps Ethernet packet pro- cessing because it is constrained by the iterative process of the Segment Engine and the SDRAM Controller interface.

Up to now, an ASIC-based dynamic CAM can support 1.5-Million lookup entrants for 72-bit data [29], and an FPGA-based DDR3 Hash-CAM is announced for the capability of 1-Million-Entrants lookup table with 104-bit wide [30]. Future developments will focus on increasing the capacity of our TCAM and improving the system's throughput. An advanced DDR-TCAM with a DDR interface controller will be implemented to reduce the read and write latency of the system.

Table 4
Comparison with different FPGA-based TCAM architectures.

TCAM Architectures	Size (D × W)	Reg.	Memory (Kbits)	Speed (MHz)	Throughput (Gbps)
Pseudo-TCAM [13]	10 K × 108	5200	110	213	23.0
Z-TCAM [14]	512 × 36	665	1,474	176	6.33
E-TCAM [15]	512 × 36	521	1,474	164	5.77
REST [18]	72 × 28	390	36	35	0.98
HP-TCAM [27]	512 × 36	2,057	2,054	118	4.25
UE-TCAM [28]	512 × 36	521	1,179	202	7.26
EE-TCAM III [21]	512 × 36	712	589	276	11.8
Jiang [26]	4 K × 150	–	40,108	125	18.7
SDRAM-based Hash-CAM [23]	64 K × 32	17,312	22	200	9.3
Proposed TCAM I	64 K × 36	1,798	0	145	0.18
Proposed TCAM II	64 K × 128	3,705	0	163	0.25

In addition, improvements in the algorithm will be investigated to make TCAM more efficient and denser.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We acknowledge the support of time and facilities from Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for this study.

References

[1] Pagiamtzis K, Sheikholeslami A. Content-addressable memory (cam) cir- cuits and architectures: A tutorial and survey. IEEE J Solid State Circuits 2006;41(3):712–27. https://doi.org/10.1109/JSSC.2005.864128.
[2] Irfan M, Sanka A, Ullah Z, Cheung R. Reconfigurable content- addressable memory (cam) on fpgas: A tutorial and survey. Future Gener Comput Syst 2022;128: 451–65. https://doi.org/10.1016/j.future.2021.09.037.
[3] Sun Y, Kim M. Ip prefix matching with binary and ternary cams. In: 2010 7th IEEE consumer communications and networking conference, 2010, p. 1–2. doi:10.1109/CCNC.2010.5421680.
[4] Ray S, Ghosh S. Smart ternary content addressable memory (stcam) archi- tecture. In: 2012 IEEE International Conference on Advanced Communi- Cation Control and Computing Technologies (ICACCCT; 2012. p. 434–8. https://doi.org/10.1109/ICACCCT.2012.6320817.
[5] Fang Y-T, Huang T-C, Wang P-C. Ternary cam compaction for ip ad- dress lookup. In: 22nd International Conference on Advanced Information Networking and Applications - Workshops (aina workshops 2008; 2008. p. 1462–7. https://doi.org/10.1109/WAINA.2008.168.
[6] Suresh K, Ramana D. Power reduction in ternary cam with pre-charge controller. In: 2018 international conference on inventive research in computing applications (ICIRCA, 2018, p. 1089–1093. doi:10.1109/ICIRCA. 2018.8597251.
[7] Irfan M, Ullah Z, Cheung R. D-tcam: A high-performance distributed ram based tcam architecture on fpgas. IEEE Access 2019;7:96060–9. https://doi.org/10.1109/ACCESS.2019.2927108.
[8] Karam R, Puri R, Ghosh S, Bhunia S. Emerging trends in design and applications of memory-based computing and content-addressable memo- ries. Proc IEEE 2015; 103(8):1311–30. https://doi.org/10.1109/JPROC.2015. 2434888.
[9] Ullah A, Reviriego P, Sánchez-Macían A, Maestro J. Multiple cell upset injection in brams for xilinx fpgas. IEEE Trans Device Mater Reliab 18 (4) (2018) 636–638. doi: 10.1109/TDMR.2018.2878806.

[10] Nikolov H, Stefanov T, Deprettere E. Efficient external memory inter- face for multi-processor platforms realized on fpga chips. International Conference on Field Programmable Logic and Applications 2007;2007:580–4. https://doi.org/10.1109/FPL.2007.4380721.
[11] Wang B, Du J, Bi X, Tian X, High bandwidth memory interface de- sign based on ddr3 sdram and fpga. International SoC Design Conference (ISOCC). Gyungju, South Korea 2015;2015:253–4. https://doi.org/10.1109/ISOCC.2015.7401743.
[12] Zhang J, Yang R, Cao X, Li J. A resource-saving tcam structure based on sram. In: 2019 IEEE 5th international conference on computer and communications (ICCC, 2019, p. 365–369. doi:10.1109/ICCC47050.2019. 9064203.
[13] Yu W, Sivakumar S, Pao D. Pseudo-tcam: Sram-based architecture for packet classification in one memory access. IEEE Netw Lett 2019;1(2):89–92. https://doi.org/10.1109/LNET.2019.2897934.
[14] Ullah Z, Jaiswal M, Cheung R. Z-tcam: An sram-based architecture for tcam. IEEE Trans Very Large Scale Integr VLSI Syst 2015;23(2):402–6. https://doi.org/10.1109/TVLSI.2014.2309350.
[15] Ullah DZ, Jaiswal M, Cheung R. Circuits syst signal process e-tcam: An efficient sram-based architecture for tcam. Circuits Syst Signal Process 2014;33. https://doi.org/10.1007/s00034-014-9796-3.
[16] Irfan M, Yantır H, Ullah Z, Cheung R. Comp-tcam: An adaptable com- posite ternary content-addressable memory on fpgas. IEEE Embed Syst Lett 2022;14(2): 63–6. https://doi.org/10.1109/LES.2021.3124747.
[17] Locke K. Parameterizable Content-Addressable Memory. Xillinx 2011.
[18] Ahmed A, Park K, Baeg S. Resource-efficient sram-based ternary content addressable memory. IEEE Trans Very Large Scale Integr VLSI Syst 2017;25(4): 1583–7. https://doi.org/10.1109/TVLSI.2016.2636294.
[19] Roth A, Foss D, McKenzie R, Perry D. Advanced ternary cam circuits on 0.13 /spl mu/m logic process technology. In: Proceedings of the IEEE 2004 Custom Integrated Circuits Conference (IEEE Cat; 2004. p. 465–8. https://doi.org/10.1109/CICC.2004.1358852.
[20] Trinh N, Kim A, Nguyen H, Tran L. Algorithmic tcam on fpga with data collision approach. Indones J Electr Eng Comput Sci 2021;22:89. https://doi.org/10.11591/ijeecs.v22.i1.pp89-96.
[21] Ullah I, Ullah DZ, Lee J. Ee-tcam: An energy-efficient sram-based tcam on fpga. Electronics 2018;7:186. https://doi.org/10.3390/electronics7090186.
[22] Reviriego P, Pontarelli S, Ullah A, Zahir A, Bianchi G. Multiple hash matching units (mhmu): An algorithmic ternary content addressable mem- ory design for field programmable gate arrays. In: in: 2018 IEEE 19th Inter- national Conference on High Performance Switching and Routing (HPSR; 2018. p. 1–6. https://doi.org/10.1109/HPSR.2018.8850764.
[23] Yang X, Mu J, Sezer S, McCanny J, Swartzlander E, High perfor- mance ip lookup circuit using ddr sdram. IEEE International SOC Conference. Newport Beach, CA, USA 2008;2008:371–4. https://doi.org/10.1109/SOCC.2008.4641547.
[24] cyclone® v hard processor system technical reference manual, IntelAc- cessed Aug. 19, 2022). URL https://www.intel.com/content/www/us/en/docs/programmable/683126/21-2/hard-processor-system-technical-reference. html.
[25] Sato Y, Otsuka K, Kobayashi K, Kouchi T, Uwai M, Nishizawa M. Novel approach for search engine, in: 2016 11th International Microsys- tems, Packaging, Assembly and Circuits Technology Conference (IMPACT, Taipei, Taiwan, 2016, p. 69–72. doi:10.1109/IMPACT.2016.7799977.
[26] Jiang W. Scalable ternary content addressable memory implementation us- ing fpgas. In: Architectures for Networking and Communications Systems, San Jose, CA, USA, 2013, p. 71–82. doi:10.1109/ANCS.2013.6665177.
[27] Ullah Z, Ilgon K, Baeg S. Hybrid partitioned sram-based ternary content addressable memory. IEEE Trans Circuits Syst Regul Pap 59 (12) (2012) 2969–2979,. doi:10.1109/TCSI.2012.2215736.
[28] Ullah Z, Ilgon K, Baeg S. Hybrid partitioned sram-based ternary content addressable memory. IEEE Trans Circuits Syst Regul Pap 59 (12) (2012) 2969–2979,. doi:10.1109/TCSI.2012.2215736.
[29] Hanzawa S, Sakata T, Kajigaya K, Takemura R, Kawahara T. A dynamic cam - based on a one-hot-spot block code - for millions-entry lookup, in: 2004 Symposium on VLSI Circuits. Digest of Technical Papers (IEEE Cat. No.04CH37525, 2004, p. 382–385. doi:10.1109/VLSIC.2004. 1346623.
[30] Yang X, Sezer S, McCanny J. D. Burns, Ddr3 based lookup circuit for high-performance network processing. In: 2009 IEEE International SOC Conference (SOCC, 2009, p. 351–354. doi:10.1109/SOCCON.2009. 5398024.